

N8N CONFERENCE · FEATURE KITCHEN

The 400 instances dilemma

How do you move a workflow from a safe place to production — without duplicating it, losing data, or shipping empty credentials?

~40 people · 50 minutes · we need your setups, not your applause

It worked in dev.

Friday. A builder copies an invoice workflow into production by hand. The canvas is identical. The credential has the same name. Monday, finance asks why 1,200 invoices went nowhere.

Nothing errored

The workflow ran. It just ran against the wrong account.

Nobody reviewed it

There was no diff to review. There was a copy.

No way back

The dev version had already moved on.

House rule: interrupt. If a model in here would fail in your org, say it in the room.

400

is not a joke. it is a topology.

Stages

dev / test / staging / prod. isolation first, then promotion.

Blast radius

a bad workflow must not touch live money, PII, or customers.

Tenants & regions

one template, many copies: country, brand, business unit.

Trust boundaries

prod is locked. credentials never leave. networks do not talk.

Hands: 1 instance · 2-5 · 6-20 · 20+ · “don’t ask”

Manual promotion fails in three boring ways

DUPLICATES

Two workflows, one name

Copy-paste between instances. IDs drift. Nobody knows which one is true.

DATA LOSS

Overwrite vs skip

Full import stomps prod edits. Partial copy leaves nodes, folders, and data tables behind.

CREDENTIALS

It looks ready

The graph arrives. The secrets do not. First prod execution is the test.

The bar is not “it copied.” The bar is **I trust this will work in production.**

WHAT PEOPLE DO TODAY

Two workarounds. both are load-bearing.

SOURCE CONTROL

Git as the pipe

Each instance is an environment. Push from dev, pull on prod. Workflows, projects, folders, tags, variables, data tables, credential *stubs* — not secrets.

INSTANCE IMPORT

Move the whole world

Export / import an instance (or a pile of workflows) when you bootstrap, migrate, or recover. Poor as a daily promote button.

Git is already the closest thing we have to a promotion API. The UX is still “you are a Git operator.”

Four promotion models

FITS MANY ORGS

1. Git is the source of truth

Instances never talk. GitHub / GitLab / Bitbucket (or any Git remote) holds the artifacts. Review, audit, rollback come for free.

NEW UX ON 1

2. Instance → instance through a provider

You click Promote. n8n still writes Git under the hood. The provider is the trust and audit layer.

DROPPED FOR NOW

3. Instance → instance direct

Prod opens a hole for another n8n. Auth, network, confused deputy, “who can push live graphs?”

MAYBE LATER

4. Environments inside one instance

One URL, many stages. Elegant for small teams. Painful migration, shared blast radius, credential isolation is the whole product.

WHICH MODEL FITS WHICH TEAM

Honest scorecard

YOUR SITUATION	GIT IS TRUTH	PROMOTE VIA PROVIDER	DIRECT INSTANCE SYNC	ENVS IN ONE INSTANCE
Dev → prod, org already lives in Git	works today	same, nicer	not needed	loses isolation
Builders who must never see Git	UX wall	this is the point	simple, risky	appealing
Production is air-gapped from Git	blocked	blocked	the only fit	works, one box
One template → many tenant instances	template yes, forks no	same gap	400 open doors	isolation
Two builders, one workflow, same day	conflicts, manual	unsolved	no review	no branches

Row 3 is the uncomfortable one. **The model we dropped is the only one that fits it.** Tell us if you are row 3.

THE DIRECTION

Promote in n8n. Persist in Git.

You stay in n8n

Pick workflows (and friends). Choose the target environment. See a diff. Confirm.

Git still exists

Branch, PR, history, CODEOWNERS, your existing controls. We do not replace your platform team.

Secrets stay put

Credential shape and IDs can travel. Values do not. Variables and external secrets fill the gap per instance.

If Git is banned in your prod network, say so later. That constraint kills this model.

WHY NOT THE OTHER TWO

Direct sync and “envs in one box”

Direct instance talk

- Prod must accept writes from another server
- Identity: user? instance? CI job?
- No natural review surface
- One stolen token promotes everything

Environments in one instance

- Customers already split instances for a reason
- Hard to migrate 400 boxes into one mental model
- Isolation (data, credentials, RBAC) becomes our problem overnight
- Still useful later for simpler teams

The workflow that looks ready

Dev Slack OAuth $\neq$ prod Slack workspace. Git syncs name, type, and id. Secrets stay on each instance. The canvas is identical. The first prod run posts to nowhere — or fails.

A. Block promote

Refuse until every credential on the target is mapped and tested.

B. Promote anyway

Ship the graph. Mark missing secrets. Let ops fill them in.

C. Copy secrets

Never, sometimes, only with a vault, only same org?

2 minutes, tables: what would you trust on a payroll workflow?

What did we actually change?

A workflow file is a large JSON graph. Git will say 400 lines changed. The builder changed one HTTP URL and dragged a node.

Text diff

Honest, noisy, scares reviewers, misses “this node moved 8px.”

Node-aware diff

Added / removed / param changed. We must define equality. Positions? notes? pin data?

Behavioral diff

Same graph, different outcome: credentials, variables, data tables, pinned data.

Vote: I need to approve a *picture* · a *param list* · a *test run on prod-like data*

Two builders. 400 copies.

Conflict

A and B both edit the same node on different branches. JSON merge is roulette. Last-write-wins silently drops work. A lock freezes the team.

Fleet

One global template, 400 regional instances, local forks for tax rules. Promote-all stamps over forks — or drift lives forever.

Pick one pain to solve first: two-person merge · branch-per-env · template vs 400 forks

DO THIS TODAY

What we would run if it were our payroll

Now

- Separate instances for isolation
- Git between them (source control)
- Protect production (read-only building)
- One branch per environment, keep it boring
- Variables + external secrets; never Git the keys
- Instance import for bootstrap / DR, not daily ship

Next

- Promote as a first-class action, Git underneath
- Workflow-aware diffs, not 2k-line JSON
- Credential mapping you can trust before go-live
- A clear conflict story (and what we will not auto-merge)
- Later, maybe: stages inside one instance for simpler teams

Your instance count is a design input.

Find us after. Bring the weird topology. We will trade you the unsolved merge.

Tell us

how many instances, and what each one means

Tell us

who is allowed to click promote

Tell us

whether production can reach Git

Tell us

the promote bug that woke you up